

MINGW-packages for Electronic Design Automation (EDA)

MINGW-packages for EDA

- Context
- Packages/tools
- Usage
 - Installation
 - Individual packages
 - Package group `eda`
- Contributing
 - Testing
 - Updating packages
 - Adding new packages
 - Testing PKGBUILD recipes downstream
- References

 [hdl MINGW-packages](https://github.com/hdl/MINGW-packages) (<https://github.com/hdl/MINGW-packages>)

 [hdl packages](https://github.com/hdl/packages) (<https://github.com/hdl/packages>)

 [chat on gitter](https://gitter.im/hdl/community) (<https://gitter.im/hdl/community>)

 [hdl awesome](https://github.com/hdl/awesome) (<https://github.com/hdl/awesome>)

 [hdl constraints](https://github.com/hdl/constraints) (<https://github.com/hdl/constraints>)

 [hdl smoke-tests](https://github.com/hdl/smoke-tests) (<https://github.com/hdl/smoke-tests>)

This repository contains references to package recipes (PKGBUILD files) for [electronic design automation \(EDA\)](https://en.wikipedia.org/wiki/Electronic_design_automation) tools/projects to be built as MinGW-w64 targets on [MSYS2](https://www.msys2.org/) (MINGW32 and MINGW64). The main purpose is coordination of contributions for upstreaming all tools/projects to official MSYS2 repositories, and having them updated periodically. See Context for a discussion about other packaging alternatives for EDA tooling.

Context

Multiple open source and/or free EDA tools are developed and primarily used on GNU/Linux platforms. Typically, builds are managed with tools such as `make` or `cmake`, and GCC or LLVM/Clang are used for compiling. Migrating all those build systems to *native* Windows toolchains can be painful. Using PKGBUILD files for MINGW32/MINGW64 allows reusing the existing plumbing and, at the same time, produced binaries can be used on Windows natively. MSYS2 is the most used environment for building open source EDA tools already; however, most projects don't use PKGBUILD recipes but custom scripts. Hence, the work on this project is mainly rewriting custom scripts for adhering to MSYS2 packaging guidelines.

Overall, there are a handful of approaches for providing *great prepackaged/prebuilt and easy-to-set-up EDA tooling environments*: see [hdl/packages](https://github.com/hdl/packages).

Packages/tools

The complete list of available MSYS2 packages is shown at packages.msys2.org/package. When first upstreamed, packages are shown at packages.msys2.org/new until maintainers of MSYS2 sign them and move them to the official repositories. By the same token, when updates or patches are upstreamed, packages are shown at packages.msys2.org/queue until signed and uploaded. Therefore, users and contributors should expect some delay between updates being accepted in [msys2/MINGW-packages](https://github.com/msys2/MINGW-packages) and changes being available in official repositories.

The following tables summarise the state ( upstreamed,  ready but not upstreamed,  work in progress,  to do) of EDA related packages, and provide references to relevant issues/PRs regarding their support on MSYS2.

Tool	State	PKGBUILD (Packages)	PRs	References	open-tool-chain (https://github.com/hdl/open-tool-chain) (static)
BitMan (https://github.com/khoapham/bitman)					
Boolector (https://hdl.github.io/awesome/items/boolector)					
Fritzing (https://hdl.github.io/awesome/items/fritzing)		mingw-w64-fritzing (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-fritzing) x86_64 (https://packages.msys2.org/package/mingw-w64-x86_64-fritzing))			

Tool	State	PKGBUILD (Packages)	PRs	References	open-toolchain (https://github.com/username/toolchain) (https://github.com/username/toolchain) (static)
GHDL (https://hdl.github.io/awesome/items/ghdl)		mingw-w64-ghdl (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-ghdl) (i686 (https://packages.msys2.org/package/mingw-w64-i686-ghdl-mcode) , x86_64 (https://packages.msys2.org/package/mingw-w64-x86_64-ghdl-llvm))	#5757 (https://github.com/msys2/MINGW-packages/pull/5757) #6688 (https://github.com/msys2/MINGW-packages/pull/6688) #7510 (https://github.com/msys2/MINGW-packages/pull/7510) #7606 (https://github.com/msys2/MINGW-packages/pull/7606) #7685 (https://github.com/msys2/MINGW-packages/pull/7685) #7817 (https://github.com/msys2/MINGW-packages/pull/7817) #7880 (https://github.com/msys2/MINGW-packages/pull/7880) #8180 (https://github.com/msys2/MINGW-packages/pull/8180) #8260 (https://github.com/msys2/MINGW-packages/pull/8260) #8451 (https://github.com/msys2/MINGW-packages/pull/8451) #8621 (https://github.com/msys2/MINGW-packages/pull/8621) #8934 (https://github.com/msys2/MINGW-packages/pull/8934) #9172 (https://github.com/msys2/MINGW-packages/pull/9172) #9468 (https://github.com/msys2/MINGW-packages/pull/9468) #9495 (https://github.com/msys2/MINGW-packages/pull/9495) #9597 (https://github.com/msys2/MINGW-packages/pull/9597) #9679 (https://github.com/msys2/MINGW-packages/pull/9679) #9790 (https://github.com/msys2/MINGW-packages/pull/9790) #9997 (https://github.com/msys2/MINGW-packages/pull/9997) #10198 (https://github.com/msys2/MINGW-packages/pull/10198) #10449 (https://github.com/msys2/MINGW-packages/pull/10449) #10548 (https://github.com/msys2/MINGW-packages/pull/10548) #11397		

Tool	State	PKGBUILD (Packages)	PRs	References	open-toolchain (https://github.com/open-toolchain/open-toolchain) (static)
			https://github.com/msys2/MINGW-packages/pull/11397 #11897 https://github.com/msys2/MINGW-packages/pull/11897 #12901 https://github.com/msys2/MINGW-packages/pull/12901 #13819 https://github.com/msys2/MINGW-packages/pull/13819 #15939 https://github.com/msys2/MINGW-packages/pull/15939		
ghdl-yosys-plugin (https://hdl.github.io/awesome/items/ghdl-yosys-plugin)	■	Built in yosys (see below)	-		✓
Graphviz (https://hdl.github.io/awesome/items/graphviz)	■	mingw-w64-graphviz (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-graphviz) x86_64 (https://packages.msys2.org/package/mingw-w64-x86_64-graphviz))			?
GTKWave (https://hdl.github.io/awesome/items/gtkwave)	■	mingw-w64-gtkwave (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-gtkwave) x86_64 (https://packages.msys2.org/package/mingw-w64-x86_64-gtkwave))	#7055 https://github.com/msys2/MINGW-packages/pull/7055 #7158 https://github.com/msys2/MINGW-packages/pull/7158 #7671 https://github.com/msys2/MINGW-packages/pull/7671 #8620 https://github.com/msys2/MINGW-packages/pull/8620 #8669 https://github.com/msys2/MINGW-packages/pull/8669 #8987 https://github.com/msys2/MINGW-packages/pull/8987 #9597 https://github.com/msys2/MINGW-packages/pull/9597		?
icestorm (https://hdl.github.io/awesome/items/icestorm)	■	mingw-w64-icestorm (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-icestorm) x86_64 (https://packages.msys2.org/package/mingw-w64-x86_64-icestorm))	#7304 https://github.com/msys2/MINGW-packages/pull/7304 #8180 https://github.com/msys2/MINGW-packages/pull/8180 #9597 https://github.com/msys2/MINGW-packages/pull/9597 #11397 https://github.com/msys2/MINGW-packages/pull/11397		✓

Tool	State	PKGBUILD (Packages)	PRs	References	open-toolchain (https://github.com/steveicarus/open-toolchain) (static)
iverilog (https://hdl.github.io/awesome/items/iverilog)		mingw-w64-iverilog (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-iverilog) (x86_64) (https://packages.msys2.org/package/mingw-w64-x86_64-iverilog))	#7341 (https://github.com/msys2/MINGW-packages/pull/7341) #7401 (https://github.com/msys2/MINGW-packages/pull/7401) #7967 (https://github.com/msys2/MINGW-packages/pull/7967) #8180 (https://github.com/msys2/MINGW-packages/pull/8180) #9172 (https://github.com/msys2/MINGW-packages/pull/9172) #9444 (https://github.com/msys2/MINGW-packages/pull/9444) #9495 (https://github.com/msys2/MINGW-packages/pull/9495) #9597 (https://github.com/msys2/MINGW-packages/pull/9597) #9997 (https://github.com/msys2/MINGW-packages/pull/9997) #10198 (https://github.com/msys2/MINGW-packages/pull/10198) #10505 (https://github.com/msys2/MINGW-packages/pull/10505) #11397 (https://github.com/msys2/MINGW-packages/pull/11397)	#398 (https://github.com/steveicarus/iverilog/pull/398)	?
ImHex (https://github.com/WerWolv/ImHex)		umarcor/MINGW-packages@imhex: mingw-w64-imhex (https://github.com/umarcor/MINGW-packages/tree/master/mingw-w64-imhex) umarcor/MINGW-packages@imhex-1.6.1: mingw-w64-imhex (https://github.com/umarcor/MINGW-packages/tree/master/mingw-w64-imhex)			?
KiCad (https://hdl.github.io/awesome/items/kicad)		mingw-w64-kicad (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-kicad) (x86_64) (https://packages.msys2.org/package/mingw-w64-x86_64-kicad))	#7444 (https://github.com/msys2/MINGW-packages/pull/7444) #8622 (https://github.com/msys2/MINGW-packages/pull/8622) #8897 (https://github.com/msys2/MINGW-packages/pull/8897)		?
KLayout (https://hdl.github.io/awesome/items/klayout)				#755 (https://github.com/KLayout/klayout/issues/755)	?

Tool	State	PKGBUILD (Packages)	PRs	References	open-toolchain (https://github.com/nextpnr/nextpnr) (static)
nextpnr (https://hdl.github.io/awesome/items/nextpnr)	■	mingw-w64-nextpnr (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-nextpnr) x86_64 (https://packages.msys2.org/package/mingw-w64-x86_64-nextpnr))	#7542 (https://github.com/msys2/MINGW-packages/pull/7542) #8231 (https://github.com/msys2/MINGW-packages/pull/8231) #8956 (https://github.com/msys2/MINGW-packages/pull/8956) #9172 (https://github.com/msys2/MINGW-packages/pull/9172) #9444 (https://github.com/msys2/MINGW-packages/pull/9444) #9495 (https://github.com/msys2/MINGW-packages/pull/9495) #9597 (https://github.com/msys2/MINGW-packages/pull/9597) #9679 (https://github.com/msys2/MINGW-packages/pull/9679) #9790 (https://github.com/msys2/MINGW-packages/pull/9790) #9997 (https://github.com/msys2/MINGW-packages/pull/9997) #10198 (https://github.com/msys2/MINGW-packages/pull/10198) #10282 (https://github.com/msys2/MINGW-packages/pull/10282) #10449 (https://github.com/msys2/MINGW-packages/pull/10449) #10505 (https://github.com/msys2/MINGW-packages/pull/10505) #11468 (https://github.com/msys2/MINGW-packages/pull/11468)	#546 (https://github.com/YosysHQ/nextpnr/issues/546)	✓
ngspice (https://hdl.github.io/awesome/items/ngspice)	■	mingw-w64-ngspice (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-ngspice) x86_64 (https://packages.msys2.org/package/mingw-w64-x86_64-ngspice))	#7450 (https://github.com/msys2/MINGW-packages/pull/7450) #7503 (https://github.com/msys2/MINGW-packages/pull/7503) #7867 (https://github.com/msys2/MINGW-packages/pull/7867) #9442 (https://github.com/msys2/MINGW-packages/pull/9442) #9447 (https://github.com/msys2/MINGW-packages/pull/9447) #11648 (https://github.com/msys2/MINGW-packages/pull/11648)		?

Tool	State	PKGBUILD (Packages)	PRs	References	open-toolchain (https://github.com/azonenberg/open-toolchain) (static)
prjtrellis (https://hdl.github.io/awesome/items/prjtrellis)		mingw-w64-prjtrellis (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-prjtrellis) (x86_64) (https://packages.msys2.org/package/mingw-w64-x86_64-prjtrellis))	#7509 (https://github.com/msys2/MINGW-packages/pull/7509) #8180 (https://github.com/msys2/MINGW-packages/pull/8180) #8414 (https://github.com/msys2/MINGW-packages/pull/8414) #8466 (https://github.com/msys2/MINGW-packages/pull/8466) #8956 (https://github.com/msys2/MINGW-packages/pull/8956) #9172 (https://github.com/msys2/MINGW-packages/pull/9172) #9597 (https://github.com/msys2/MINGW-packages/pull/9597) #10449 (https://github.com/msys2/MINGW-packages/pull/10449) #10573 (https://github.com/msys2/MINGW-packages/pull/10573) #11435 (https://github.com/msys2/MINGW-packages/pull/11435)		
scopehal (https://hdl.github.io/awesome/items/scopehal)		mingw-w64-scopehal-apps (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-scopehal-apps) (x86_64) (https://packages.msys2.org/package/mingw-w64-x86_64-scopehal-apps))	#8676 (https://github.com/msys2/MINGW-packages/pull/8676) #8972 (https://github.com/msys2/MINGW-packages/pull/8972) #9172 (https://github.com/msys2/MINGW-packages/pull/9172) #9444 (https://github.com/msys2/MINGW-packages/pull/9444) #9485 (https://github.com/msys2/MINGW-packages/pull/9485) #9524 (https://github.com/msys2/MINGW-packages/pull/9524) #9997 (https://github.com/msys2/MINGW-packages/pull/9997) #10198 (https://github.com/msys2/MINGW-packages/pull/10198)	#388 (https://github.com/azonenberg/scopehal-apps/pull/388)	

Tool	State	PKGBUILD (Packages)	PRs	References	open-toolchain (https://github.com/tool-forge/fjtoolchain) (static)
Serial-Studio (https://hdl.github.io/awesome/items/serial-studio)		mingw-w64-serial-studio (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-serial-studio) (x86_64 (https://packages.msys2.org/package/mingw-w64-x86_64-serial-studio))	#7682 (https://github.com/msys2/MINGW-packages/pull/7682) #7845 (https://github.com/msys2/MINGW-packages/pull/7845) #7964 (https://github.com/msys2/MINGW-packages/pull/7964) #8180 (https://github.com/msys2/MINGW-packages/pull/8180) #9532 (https://github.com/msys2/MINGW-packages/pull/9532) #10198 (https://github.com/msys2/MINGW-packages/pull/10198)		?
sigrok-cli (https://hdl.github.io/awesome/items/sigrok-cli)					?
Surelog (https://hdl.github.io/awesome/items/surelog)		umarcor/MINGW-packages@UHDM-Surelog: mingw-w64-surelog (https://github.com/umarcor/MINGW-packages/tree/UHDM-Surelog/mingw-w64-surelog)			?
SymbiYosys (https://hdl.github.io/awesome/items/symbiyosys)					✓
teuchos/trilinos (https://trilinos.github.io/teuchos.html)					?
UHDM (https://hdl.github.io/awesome/items/uhtml)		umarcor/MINGW-packages@UHDM-Surelog: mingw-w64-UHDM (https://github.com/umarcor/MINGW-packages/tree/UHDM-Surelog/mingw-w64-UHDM)			?
Verible (https://hdl.github.io/awesome/items/verible)					?

Tool	State	PKGBUILD (Packages)	PRs	References	open-toolchain (https://github.com/open-toolchain/open-toolchain) (static)
Verilator (https://hdl.github.io/awesome/items/verilator)		mingw-w64-verilator (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-verilator) (x86_64) (https://packages.msys2.org/package/mingw-w64-x86_64-verilator))	#7424 (https://github.com/msys2/MINGW-packages/pull/7424) #7872 (https://github.com/msys2/MINGW-packages/pull/7872) #8180 (https://github.com/msys2/MINGW-packages/pull/8180) #8465 (https://github.com/msys2/MINGW-packages/pull/8465) #8954 (https://github.com/msys2/MINGW-packages/pull/8954) #9111 (https://github.com/msys2/MINGW-packages/pull/9111) #9790 (https://github.com/msys2/MINGW-packages/pull/9790) #10282 (https://github.com/msys2/MINGW-packages/pull/10282) #10587 (https://github.com/msys2/MINGW-packages/pull/10587) #11247 (https://github.com/msys2/MINGW-packages/pull/11247) #11898 (https://github.com/msys2/MINGW-packages/pull/11898) #12900 (https://github.com/msys2/MINGW-packages/pull/12900) #13820 (https://github.com/msys2/MINGW-packages/pull/13820) #14598 (https://github.com/msys2/MINGW-packages/pull/14598) #15350 (https://github.com/msys2/MINGW-packages/pull/15350)	#2681 (https://github.com/verilator/verilator/pull/2681) #2685 (https://github.com/verilator/verilator/pull/2685) #3471 (https://github.com/verilator/verilator/pull/3471)	
VHDLproc (https://github.com/nobodywasishere/VHDLproc)					
VPR (https://hdl.github.io/awesome/items/vpr)					
VTR (https://hdl.github.io/awesome/items/vtr)					
Xyce (https://hdl.github.io/awesome/items/xyce)					
Yices 2 (https://hdl.github.io/awesome/items/yices2)		mingw-w64-yices (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-yices) (x86_64) (https://packages.msys2.org/package/mingw-w64-x86_64-yices))			

Tool	State	PKGBUILD (Packages)	PRs	References	open-toolchain (https://github.com/ToolForge/fjtoolchain) (static)
Yosys (https://hdl.github.io/awesome/items/yosys)		mingw-w64-yosys (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-yosys) x86_64 (https://packages.msys2.org/package/mingw-w64-x86_64-yosys))	#7534 (https://github.com/msys2/MINGW-packages/pull/7534) #7606 (https://github.com/msys2/MINGW-packages/pull/7606) #7965 (https://github.com/msys2/MINGW-packages/pull/7965) #8180 (https://github.com/msys2/MINGW-packages/pull/8180) #8290 (https://github.com/msys2/MINGW-packages/pull/8290) #8956 (https://github.com/msys2/MINGW-packages/pull/8956) #9172 (https://github.com/msys2/MINGW-packages/pull/9172) #9444 (https://github.com/msys2/MINGW-packages/pull/9444) #9495 (https://github.com/msys2/MINGW-packages/pull/9495) #9597 (https://github.com/msys2/MINGW-packages/pull/9597) #9679 (https://github.com/msys2/MINGW-packages/pull/9679) #9790 (https://github.com/msys2/MINGW-packages/pull/9790) #9997 (https://github.com/msys2/MINGW-packages/pull/9997) #10198 (https://github.com/msys2/MINGW-packages/pull/10198) #10449 (https://github.com/msys2/MINGW-packages/pull/10449)	#2505 (https://github.com/YosysHQ/yosys/issues/2505) #2507 (https://github.com/YosysHQ/yosys/pull/2507) #2511 (https://github.com/YosysHQ/yosys/pull/2511) #2512 (https://github.com/YosysHQ/yosys/pull/2512) #2514 (https://github.com/YosysHQ/yosys/pull/2514) #2515 (https://github.com/YosysHQ/yosys/pull/2515) #13821 (https://github.com/msys2/MINGW-packages/pull/13821)	
Z3 (https://hdl.github.io/awesome/items/z3)		mingw-w64-z3 (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-z3) x86_64 (https://packages.msys2.org/package/mingw-w64-x86_64-z3))			

Board programming	State	PKGBUILD (Packages)	PRs	References	!
					!
					!
dfu-util (https://hdl.github.io/awesome/items/dfu-util)	■	mingw-w64-dfu-util (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-dfu-util) (x86_64) (https://packages.msys2.org/package/mingw-w64-x86_64-dfu-util))	#7400 (https://github.com/msys2/MINGW-packages/pull/7400)		
ecpprog (https://hdl.github.io/awesome/items/ecpprog)	■	mingw-w64-ecpprog (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-ecpprog) (x86_64) (https://packages.msys2.org/package/mingw-w64-x86_64-ecpprog))	#7349 (https://github.com/msys2/MINGW-packages/pull/7349) #7403 (https://github.com/msys2/MINGW-packages/pull/7403) #9172 (https://github.com/msys2/MINGW-packages/pull/9172)		
fujprog (https://hdl.github.io/awesome/items/fujprog)	■	umarcor/MINGW-packages@fujprog; mingw-w64-fujprog (https://github.com/umarcor/MINGW-packages/tree/fujprog/mingw-w64-fujprog)			
icesprog (https://hdl.github.io/awesome/items/icesprog)	■	mingw-w64-icesprog (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-icesprog) (x86_64) (https://packages.msys2.org/package/mingw-w64-x86_64-icesprog))	#7308 (https://github.com/msys2/MINGW-packages/pull/7308) #7352 (https://github.com/msys2/MINGW-packages/pull/7352) #7966 (https://github.com/msys2/MINGW-packages/pull/7966)		

Board programming	State	PKGBUILD (Packages)	PRs	References
openFPGAloader (https://hdl.github.io/awesome/items/openfpga-loader)	■	mingw-w64-openFPGAloader (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-openFPGAloader) (x86_64 (https://packages.msys2.org/package/mingw-w64-x86_64-openFPGAloader))	#7351 (https://github.com/msys2/MINGW-packages/pull/7351) #7404 (https://github.com/msys2/MINGW-packages/pull/7404) #7641 (https://github.com/msys2/MINGW-packages/pull/7641) #7971 (https://github.com/msys2/MINGW-packages/pull/7971) #8300 (https://github.com/msys2/MINGW-packages/pull/8300) #8965 (https://github.com/msys2/MINGW-packages/pull/8965) #8995 (https://github.com/msys2/MINGW-packages/pull/8995) #9160 (https://github.com/msys2/MINGW-packages/pull/9160) #9280 (https://github.com/msys2/MINGW-packages/pull/9280) #10224 (https://github.com/msys2/MINGW-packages/pull/10224) #10282 (https://github.com/msys2/MINGW-packages/pull/10282) #10449 (https://github.com/msys2/MINGW-packages/pull/10449) #11213 (https://github.com/msys2/MINGW-packages/pull/11213) #14686 (https://github.com/msys2/MINGW-packages/pull/14686)	#65 (https://github.com/trabucayre/openFPGAloader/pull/65)
OpenOCD (https://hdl.github.io/awesome/items/openocd)	■	mingw-w64-openocd (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-openocd) (x86_64 (https://packages.msys2.org/package/mingw-w64-x86_64-openocd))		

Usage

“MSYS2 (...) provides a native build environment, based on open source software, and makes you feel right at home when you are already comfortable with Linux.

— [msys2.org: What is MSYS2?](https://www.msys2.org/docs/what-is-msys2/) (https://www.msys2.org/docs/what-is-msys2/)

“In MSYS2 there are 2 types of packages:

- MSYS packages - these run on the emulation layer and are typically POSIX-only programs.
- MINGW packages - these run natively just like any other Windows program.

You should think of these two systems as separate where msys packages should generally only be build dependencies of mingw packages.

— [msys2.org: Which subsystem?](https://www.msys2.org/wiki/Creating-Packages/#which-subsystem) (https://www.msys2.org/wiki/Creating-Packages/#which-subsystem)

“There are 3 package repositories, **msys2**, **mingw32**, and **mingw64**. The packages in **msys2** are named just like on a Linux distribution, the packages in **mingw** are prefixed by either `mingw-w64-i686-` for 32-bit packages, or `mingw-w64-x86_64-` for 64-bit packages.

— [msys2.org: Package repositories](https://www.msys2.org/docs/package-management/#package-repositories) (https://www.msys2.org/docs/package-management/#package-repositories)

That is, projects/tools on MSYS2 feel like Unix when it comes to building them (GCC, clang, make, cmake, etc.), but the artifacts are native Windows binaries (“just like any other Windows program”). Typically, users want to install `mingw-w64-x86_64-` packages, which can be used outside of an MSYS2 environment optionally.



See how to launch MSYS2 shells/terminals or how to use *Windows Terminal* at [msys2.org: Terminals](https://www.msys2.org/docs/terminals/) (https://www.msys2.org/docs/terminals/).



[msys2.org: Avoiding writing long package names](https://www.msys2.org/docs/package-management/#avoiding-writing-long-package-names) (https://www.msys2.org/docs/package-management/#avoiding-writing-long-package-names)

Installation

MSYS2 can be downloaded as a tarball or as a self-extracting installer with an optional GUI. Nevertheless, both of them produce the same result because MSYS2’s installations are portable by default; i.e. they don’t create any environment variables or registry entries on the system. For this same reason, it’s possible to have several MSYS2 installations at the same time, without them colliding. Find the latest installers/tarballs at github.com/msys2/msys2-installer/releases (https://github.com/msys2/msys2-installer/releases) and follow the guidelines at [msys2.org/#installation](https://www.msys2.org/#installation) (https://www.msys2.org/#installation).

MSYS2 borrows the package management plumbing from [Arch Linux](https://www.archlinux.org/) (https://www.archlinux.org/) (see [msys2.org: MSYS2 vs Arch Linux](https://www.msys2.org/docs/what-is-msys2/#msys2-vs-arch-linux) (https://www.msys2.org/docs/what-is-msys2/#msys2-vs-arch-linux)). The default package manager is `pacman`, which is the same codebase as Arch’s but ported/patched to work on MSYS2 (Windows). Guidelines for MSYS2’s `pacman` are available at [msys2.org: Package Management](https://www.msys2.org/docs/package-management/) (https://www.msys2.org/docs/package-management/). Nonetheless, guidelines at wiki.archlinux.org/index.php/pacman (https://wiki.archlinux.org/index.php/pacman) and [wiki.archlinux.org/index.php/pacman/Tips and tricks](https://wiki.archlinux.org/index.php/pacman/Tips_and_tricks) (https://wiki.archlinux.org/index.php/pacman/Tips_and_tricks) can also be useful.

Individual packages

All upstreamed EDA packages are available through the default package manager (`pacman`), as any other regular MSYS2 package. Hence, you can search them with `pacman -Ss TOOL_NAME` (see [msys2.org: Finding a package](https://www.msys2.org/docs/package-management/#finding-a-package) (https://www.msys2.org/docs/package-management/#finding-a-package)) and then install them with `pacman -S PACKAGE_NAME` (see [msys2.org: Installing a package](https://www.msys2.org/docs/package-management/#installing-a-package) (https://www.msys2.org/docs/package-management/#installing-a-package)). Find further info at [msys2.org/docs/package-management](https://www.msys2.org/docs/package-management/) (https://www.msys2.org/docs/package-management).

Package group `eda`

Alternatively, there is a [package group](https://wiki.archlinux.org/index.php/Meta_package_and_package_group) (https://wiki.archlinux.org/index.php/Meta_package_and_package_group) named `eda`, which you can install with `pacman -S mingw-w64-x86_64-eda` (64 bits) or `pacman -S mingw-w64-i686-eda` (32 bits). The list of tools will be shown, for you to choose installing all of them or some only. See [mingw-w64-x86_64-eda](https://packages.msys2.org/group/mingw-w64-x86_64-eda) (https://packages.msys2.org/group/mingw-w64-x86_64-eda) and/or [mingw-w64-i686-eda](https://packages.msys2.org/group/mingw-w64-i686-eda) (https://packages.msys2.org/group/mingw-w64-i686-eda).

Contributing

The official git repository for PKGBUILD recipes targeting MINGW32 | MINGW64 (MSYS2) is [msys2/MINGW-packages](https://github.com/msys2/MINGW-packages) (https://github.com/msys2/MINGW-packages). Due to GitHub’s permission management constraints, it is undesirable to open Pull Request (PRs) across organisations. Therefore, all contributions regarding PKGBUILD recipes are handled in repositories from users’/contributors’ namespaces and opened against `msys2/MINGW-packages`. That allows MSYS2 maintainers to edit PRs in-place. Anyway, `hdl/MINGW-packages` can be used for discussion and coordination of tightly coupled EDA packages.

Other than that, branch `main` contains testing and tracking resources, on top of MSYS2’s infrastructure:

- On MSYS2’s CI, packages are built with option `--nocheck`. Therefore, only successful builds are checked, not runtime dependencies or other possible broken artifacts. In this repo, EDA packages are installed and tested every day.
- On MSYS2’s workflow, packages are neither rebuilt nor updated periodically. They are checked and bumped upon users’/contributors’ demand. Therefore, it is easy for less used packages to become either outdated or broken. In this repo, EDA packages are built weekly, in order to avoid them rotting.



We’d like to provide additional features, such as building EDA packages from git heads, as a complement to tagged/fixed builds; or automatically checking when are new releases available. Should be willing to contribute, [get in touch](https://gitter.im/hdl/community) (https://gitter.im/hdl/community)!

MSYS2 borrows the package management plumbing from [Arch Linux](https://www.archlinux.org/) (https://www.archlinux.org/). That is, PKGBUILD recipes and `makepkg` are used. It’s the same same codebase, but patched to work on MSYS2 (Windows). Therefore, upstream repositories are different. Apart from that, the usage is the same, and PKGBUILD files for MSYS2 are very similar to the equivalent recipes for Arch Linux.

- [msys2.org](https://www.msys2.org/) (https://www.msys2.org/)
 - [Creating Packages](https://www.msys2.org/wiki/Creating-Packages/) (https://www.msys2.org/wiki/Creating-Packages/)
 - [A new package from start to finish](https://www.msys2.org/wiki/Creating-Packages/#a-new-package-from-start-to-finish) (https://www.msys2.org/wiki/Creating-Packages/#a-new-package-from-start-to-finish)
 - [Porting](https://www.msys2.org/wiki/Porting/) (https://www.msys2.org/wiki/Porting/)
- wiki.archlinux.org (https://wiki.archlinux.org/)
 - [PKGBUILD](https://wiki.archlinux.org/index.php/PKGBUILD) (https://wiki.archlinux.org/index.php/PKGBUILD)
 - [Makepkg](https://wiki.archlinux.org/index.php/Makepkg) (https://wiki.archlinux.org/index.php/Makepkg)

Testing

Branch `main` contains subdir `test` and a Continuous Integration (CI) workflow which is scheduled periodically. There should be a test script and a job for each EDA tool. Smoke-tests from [hdl/smoke-tests](https://github.com/hdl/smoke-tests) (https://github.com/hdl/smoke-tests) are used before running heavier tests.



Tagged/fixed versions of the tools are built periodically. It would be desirable to have an scheduled workflow for building the head of each downstream repo too. Do you want to tackle the challenge? Contributions are welcome!

Updating packages

MSYS2 repositories are compared against matching packages on Arch Linux, and mismatches are shown at packages.msys2.org/outofdate (https://packages.msys2.org/outofdate). That's the easiest way for finding outdated packages. However, many EDA tools are not upstreamed to Arch Linux yet; thus, those are not shown there. It is therefore useful if frequent users of the tools watch the corresponding repositories and communicate updates either through this repository or through [msys2/MINGW-packages](https://github.com/msys2/MINGW-packages) (https://github.com/msys2/MINGW-packages).



Since most projects use source control management (SCM) tools, it should be feasible to have an script for checking new releases automatically. Should you be up to the challenge, contributions are welcome!

The process for bumping a package is very simple (when explained, at least):

- Fork and checkout the latest `master` branch from [msys2/MINGW-packages](https://github.com/msys2/MINGW-packages) (https://github.com/msys2/MINGW-packages) and create a feature branch.
- Edit the PKGBUILD recipe of the tool:
 - Change the version.
 - Reset `pkgrel`.
 - If the tool is retrieved as a tarball, update the checksum.
 - Check if any patches exist. Either remove them (if not necessary anymore), update them (if sources changed), or keep them.
- Commit and push.
- CI will run and it will build the package.

In practice, checking and updating the patches might be not so simple. Should modifying them be required, see Testing PKGBUILD recipes downstream below.

Adding new packages

All the  tools/projects in the list are to be packaged yet. Furthermore, there might be other interesting tools which you'd like to upstream, even if not listed above (yet).

TBC

On MSYS2 native windows executables (or DLLs) are built, but the paths in the (bash) shell use the UNIX syntax. Hence, typically the makefiles/scripts of the projects need to be tweaked. Sometimes there is luck, and an Arch Linux recipe works almost as is, but chances are that some defines need to be added.

[mingw-w64-PKGBUILD-templates](https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-PKGBUILD-templates) (https://github.com/msys2/MINGW-packages/tree/master/mingw-w64-PKGBUILD-templates)

Testing PKGBUILD recipes downstream

Should modifying/patching downstream sources be required, the recommended procedure is the following:

- Checkout the desired version of the project/tool repository and create a feature branch.
- Create subdir `msys2` and add the PKGBUILD recipe.
 - Remove non-external `source` entries and the corresponding checksums from the recipe.
 - Modify the paths to use the sources in the parent directory.
- Copy [testing-workflow.yml](https://github.com/hdl/MINGW-packages/tree/main/testing-workflow.yml) (https://github.com/hdl/MINGW-packages/tree/main/testing-workflow.yml) to `.github/workflows/` (create the subdir if it does not exist).
- Modify the last step in the workflow for testing the tool.
- Commit and push.
- CI will run in your fork of the downstream repo.

Build issues and regressions should arise. You can commit your fixes to the feature branch and push again. Since the workflow and the PKGBUILD recipe build the head of the branch, there is no need to generate patches yet.

TBC

- *Discuss solutions with maintainers.*
- *Propose having the MSYS2 CI added downstream.*
- *How to generate patches and apply them to the PKGBUILD recipe to be upstreamed.*

References

- [sylefeb/fpga-binutils](https://github.com/sylefeb/fpga-binutils) (https://github.com/sylefeb/fpga-binutils)
- [open-tool-forge/fpga-toolchain](https://github.com/open-tool-forge/fpga-toolchain) (https://github.com/open-tool-forge/fpga-toolchain)

- [DiodeRed/MinGW-FPGA-Packages](https://github.com/DiodeRed/MinGW-FPGA-Packages) (https://github.com/DiodeRed/MinGW-FPGA-Packages)

Last updated 2026-09-24 02:39:32 UTC